BIS 527: Introduction to Health Data Science Fall 2026

Homework 2: Shortest Paths and Sequence Alignment

Instructor: Harsh Parikh 100 points

Released: Friday, September 25, 2026
Due: Saturday, October 17, 2026 (see Canvas for submission time)

Assignment. This homework covers two problems: **shortest paths** (Problems 1–2) and **sequence alignment** (Problems 3–4). First develop each algorithm through guided questions, then prove correctness and analyze time and storage. The follow-up problems ask you to implement, test and measure the algorithms.

Submit. Upload a PDF, TeX source and Python file. Name each `hw2-`*`yourlastname`* and use the corresponding `.pdf`, `.tex` or `.py` extension. Use the supplied TeX template and include its figure files. Submit typed work only. Use the generators in `hw2-starter.py` unchanged; write your own algorithm functions. The starter also supplies interfaces and a timing helper.

Collaboration and AI. Write your own solution and code. List your collaborators. At the top, either state that you used no AI or list the tool, its use, your verification and anything left unverified. The course AI policy applies.

Rules. Use Python 3. Do not call a library routine that solves either assigned algorithm. Report software versions and run instructions. If a requested size is infeasible, reduce the largest size and explain the change. Late work is not accepted; contact the instructor before the deadline about a documented emergency.

Instructor AI declaration. OpenAI Codex assisted with drafting, reference implementations and checks. Report any error you find.

Problem 1. Shortest paths: design and analysis

25 points: five parts, 5 points each

The shortest-path problem (Problems 1–2)

Given: a directed graph $G = (V, E)$, a source vertex s, and a nonnegative integer cost $w(u, v)$ for each edge (u, v).

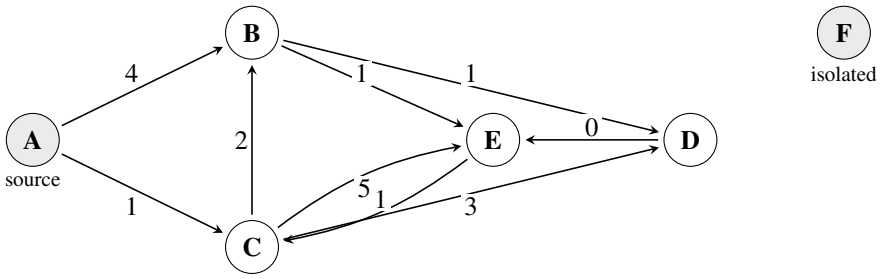
Find: a minimum-cost path from s to each reachable vertex. Follow the arrows and add the edge costs. Here "shortest" means lowest total cost, not fewest edges. For example, edge costs 2 and 3 give a path cost of 5.

Return: arrays d and p. For each vertex v, $d[v]$ is the minimum path cost and $p[v]$ is the previous vertex on one such path. Set $d[s] = 0$ and $p[s] =$ `None`. For an unreachable vertex, return $d[v] = \infty$ and $p[v] =$ `None`.

Representation. Write $N = |V|$ and $M = |E|$. The adjacency list `adj[u]` contains the outgoing pairs $(v, w(u, v))$. Zero-cost edges are allowed; self-loops and parallel edges are excluded. A route means a path throughout these questions.

Build the algorithm. During the computation, $d[v]$ is the lowest path cost found so far, and $p[v]$ records its predecessor. A *finalized* vertex will not be selected or updated again. Parts (a)–(c) develop the update and selection rules; part (d) asks you to prove that finalizing a vertex is safe.

Example graph for parts (a)–(c). Use the graph below for parts (a)–(c). The source is A, the vertex labels correspond to $A = 0, B = 1, \ldots, F = 5$, and F is isolated. These are all directed edges; each arrow label gives its cost.



(a) Develop the update rule.

1. Compute the costs of $A \to B$ and $A \to C \to B$. Can a path with fewer edges have a larger cost?
2. Suppose $d[B] = 4$, $p[B] = A$ and $d[C] = 1$. The edge $C \to B$ has cost 2. What new cost does this edge offer for B? What should $d[B]$ and $p[B]$ become?
3. Generalize: a path to u has cost $d[u]$, and (u, v) has cost $w(u, v)$. Give the candidate cost for v, the condition for replacing $d[v]$, and the new value of $p[v]$. Keep the current values when costs tie.

Problem 1, continued: construct and trace the algorithm

(b) Choose the next vertex and assemble the algorithm. After examining the edges out of A, the unfinished labels are $d[B] = 4$, $d[C] = 1$, and ∞ for D, E, F.

1. Which label, $d[B]$ or $d[C]$, can be finalized next? Use the path comparison in part (a) and the nonnegative edge costs to explain your choice.
2. State a rule for selecting the next unfinished vertex in any graph. Break ties by vertex number. State when no further vertex can be selected.
3. Complete each blank in the pseudocode using your selection and update rules.

```
d[s] = 0; d[v] = infinity for every v != s
p[v] = None and finalized[v] = False for every vertex v
for pass = 1, ..., N:
    u = ___ # choose using a full scan
    if ___: break # no finite choice remains
    finalized[u] = True
    for each (v, weight) in adj[u]:
        if not finalized[v]:
            candidate = ___
            if ___:
                d[v] = ___
                p[v] = ___
return d, p
```

(c) Trace your algorithm and recover paths. Fill both tables for the example graph. Record distances after processing each selected vertex's outgoing edges. For an unsuccessful pass, write "stop" and keep distances unchanged. Use ∞ for unreached vertices and `None` for missing predecessors.

Recover an A-to-E path from p, compute its cost and give another path of the same cost. Return paths as vertex lists including both endpoints: $[A]$ for the source and `None` for F. Explain why F is never finalized.

Pass	Selected	$d[A]$	$d[B]$	$d[C]$	$d[D]$	$d[E]$	$d[F]$
Initial	—	0	∞	∞	∞	∞	∞
1	_____	_____	_____	_____	_____	_____	_____
2	_____	_____	_____	_____	_____	_____	_____
3	_____	_____	_____	_____	_____	_____	_____
4	_____	_____	_____	_____	_____	_____	_____
5	_____	_____	_____	_____	_____	_____	_____
6	_____	_____	_____	_____	_____	_____	_____

Vertex	A	B	C	D	E	F
Final $d[v]$	_____	_____	_____	_____	_____	_____
Final $p[v]$	_____	_____	_____	_____	_____	_____

Problem 1, continued: proof and analysis

(d) Prove correctness and termination. Let $\delta(v)$ denote the true minimum path cost from s to v, or ∞ if v is unreachable. Unlike the tentative label $d[v]$, $\delta(v)$ does not change. Prove these claims.

1. Every finite $d[v]$ is the cost of a path from s to v. Check initialization and an update, and conclude that $d[v] \geq \delta(v)$.
2. Every finalized label satisfies $d[v] = \delta(v)$. Use induction on the number of finalized vertices. For the inductive step, suppose a cheaper path to the selected u exists. Consider its first unfinished vertex and the edge entering that vertex; derive a contradiction to the selection rule.
3. The algorithm stops after at most N passes. If no finite unfinished label remains, every unfinished vertex is unreachable.
4. Following predecessors from a reachable vertex and reversing the chain returns a minimum-cost path from s. Explain why the chain cannot cycle, even with zero-cost edges.

Then use $s \to a$ (cost 1), $s \to b$ (cost 2) and $b \to a$ (cost -2) to show why the second claim requires nonnegative costs.

(e) Count time and storage. Count each array access, addition and comparison as constant time. Each selection pass scans all N vertices, and each finalized vertex scans its outgoing edges once. Derive the numbers of vertex and edge inspections and the resulting time bound. Then give the adjacency-list input storage, the extra array storage and the cost of recovering one route.

Problem 2. Shortest paths: implementation and measurement

25 points: five parts, 5 points each

(a) Implement and test. Implement `minimum_cost_routes(adj, source, counts=None)`, returning (d, p), and `reconstruct_path(p, d, source, target)`, returning the vertex list from source to target. Include both endpoints; return `[source]` for the source and `None` for an unreachable target. For the example graph, `adj[0]=[(1,4),(2,1)]` lists A's outgoing edges. Use the six test inputs in the table; no other edges are present. In each expected and actual cell, give the full distance array in vertex order and the route to the specified target. Fill the expected column before running. Check each path's endpoints, directed edges and summed cost against the returned distance. Report pass/fail for each case below the table. Exclude all checks from timing.

Case	Graph; source; target	Expected	Actual
Example	Problem 1 graph; $s = A$; target E	$d =$ _____ Route: _____	$d =$ _____ Route: _____
Single vertex	$V = \{0\}$; no edges; $s = 0$; target 0	$d =$ _____ Route: _____	$d =$ _____ Route: _____
Unreachable	$V = \{0,1\}$; no edges; $s = 0$; target 1	$d =$ _____ Route: _____	$d =$ _____ Route: _____
Zero-cost cycle	$V = \{0,1,2\}$; $0 \leftrightarrow 1 \leftrightarrow 2$, all costs 0; $s = 0$; target 2	$d =$ _____ Route: _____	$d =$ _____ Route: _____
Tied routes	$V = \{0,1,2,3\}$; $0 \to 1, 0 \to 2$ cost 2; $1 \to 3, 2 \to 3$ cost 0; $s = 0$; target 3	$d =$ _____ Route: _____	$d =$ _____ Route: _____
Later improvement	$V = \{0,1,2\}$; $0 \to 1$ cost 9; $0 \to 2, 2 \to 1$ cost 1; $s = 0$; target 1	$d =$ _____ Route: _____	$d =$ _____ Route: _____

(b) Count operations. Increment `scans` for every position in each full selection scan, including finalized vertices and the final unsuccessful scan. Increment `edges` for every examined outgoing edge, including edges to finalized vertices. Counted and uncounted runs must return the same result. Enter both counts for the example graph in the table.

Counter	Observed count
`scans`	_____
`edges`	_____

Problem 2, continued: measurements

(c) Construct graph families. For $N = 32, 64, 128, 256$, use source 0 and the supplied `make_graph(N, family, seed=527)` without changes. Use `"sparse"` and `"dense"` for `family`; each call resets the seed. The sparse graph has edges $u \to (u+k) \bmod N$ for $k \in \{1,2,4\}$. The dense graph has every ordered pair of distinct vertices. Draw each weight uniformly from $\{0,\ldots,9\}$ in the generator's order. Explain reachability and complete the table. In each pred. / obs. cell, put the predicted count before the slash and the observed count after it. Report the combined entries in d, p and the finalized array.

Family	N	M	Scans pred. / obs.	Edges pred. / obs.	Total array entries
sparse	32	_____	_____/_____	_____/_____	_____
sparse	64	_____	_____/_____	_____/_____	_____
sparse	128	_____	_____/_____	_____/_____	_____
sparse	256	_____	_____/_____	_____/_____	_____
dense	32	_____	_____/_____	_____/_____	_____
dense	64	_____	_____/_____	_____/_____	_____
dense	128	_____	_____/_____	_____/_____	_____
dense	256	_____	_____/_____	_____/_____	_____

(d) Measure time. Construct inputs before timing and disable counters. Time only the call that returns distances and predecessors. Use one warmup and five batches of three calls; report the median batch mean in seconds per call. Reuse the same fixed graph for the warmup and every timed call. Record the Python version, system, seed, batch size and repetitions. Enter the times and ratios in the supplied table, using the preceding size in the same family for each ratio. Plot N on the horizontal axis and seconds per call on the vertical axis, with logarithmic scales and a labeled line for each family.

Family	N	$T(N)$ (seconds/call)	$T(N)/T(N/2)$
sparse	32	_____	—
sparse	64	_____	_____
sparse	128	_____	_____
sparse	256	_____	_____
dense	32	_____	—
dense	64	_____	_____
dense	128	_____	_____
dense	256	_____	_____

(e) Interpret the results. Use the ratios from part (d) to compare timing with the operation counts. Explain why the sparse case is not linear in N. Distinguish input storage from extra algorithm storage, and entries from bytes. Discuss one difference between count and timing trends. State one limit of drawing asymptotic conclusions from four sizes.

Problem 3. Sequence alignment: design and analysis

25 points: five parts, 5 points each

The sequence-alignment problem (Problems 3–4)

Given: two sequences $x = x_1 \cdots x_m$ and $y = y_1 \cdots y_n$ over $\{A, C, G, T\}$. Either sequence may be empty.

Find: a highest-scoring global alignment. Insert gaps (-) to make the sequences the same length, keeping every letter in its original order. No column may contain two gaps. "Global" means using both sequences in full.

Score: add the column scores: $+2$ for a match, -1 for a mismatch, and -2 for a letter paired with a gap, including gaps at either end. For inputs `AC` and `A`, the alignment `AC/A-` scores $2 - 2 = 0$.

Return: the maximum score and the two aligned sequences that attain it.

Build the algorithm. Let $F[i, j]$ be the maximum alignment score for the first i letters of x and the first j letters of y. Index 0 means an empty prefix. Parts (a)–(c) develop an algorithm by relating these smaller problems. The sequences and scores are synthetic; the score has no clinical or probabilistic interpretation.

(a) Build the recurrence from smaller problems.

1. If one prefix is empty, what must its alignment contain? Complete the boundary table to obtain $F[0,0]$, $F[i,0]$ and $F[0,j]$.
2. For $i, j > 0$, the last column has one of the three forms listed below. Remove it. Complete the remaining-prefix, column-score and candidate-score entries. Explain why there is no fourth case.
3. Combine the three candidates into a recurrence for $F[i,j]$. For `A/C`, compute $F[1,1]$ with this recurrence and then with 0 as a fourth option in the maximum. Which result satisfies the global objective? Why must boundary scores also charge for gaps?

Prefix pair	Required alignment	Best score
$(0,0)$	_____	_____
$(i,0)$	_____	_____
$(0,j)$	_____	_____

Last column	Remaining prefix pair	Column score	Candidate score
D: (x_i, y_j)	_____	_____	_____
U: $(x_i, -)$	_____	_____	_____
L: $(-, y_j)$	_____	_____	_____

Problem 3, continued: fill the table and recover an alignment

(b) Choose a fill order and work an example.

1. Which entries are needed before computing $F[i, j]$? Give a table-fill order in which those entries are already available.
2. For $x =$ `ACG` and $y =$ `AG`, complete the score table and the three candidate calculations at $(2,2)$. Rows index x; columns index y.
3. Recover an alignment by working backward from $(3,2)$ to $(0,0)$. At each cell, follow a candidate that attains its score. Use D, U and L for the diagonal, up and left cases in part (a), with ties resolved in that order. Each move emits its last-column pair. Complete the traceback table, marking unused rows with a dash. Reverse the emitted columns and score the resulting alignment.

Row i letter in x	$j=0$ empty prefix	$j=1$ A	$j=2$ G
$i=0$, empty prefix	0	_____	_____
$i=1$, A	_____	_____	_____
$i=2$, C	_____	_____	_____
$i=3$, G	_____	_____	_____

Candidate at $(2,2)$	Calculation	Value
Diagonal	_____	_____
Up	_____	_____
Left	_____	_____

Step	Current (i, j)	Move	Emitted pair	Next (i, j)
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____
5	_____	_____	_____	_____

Problem 3, continued: complete the algorithm, prove and analyze it

(c) Assemble the complete algorithm. Use a score table F and a same-sized pointer table P that stores the chosen direction D, U or L. Complete the pseudocode, including boundary scores and directions, the three candidates, tie handling, traceback moves and stopping condition. The values `cD`, `cU` and `cL` are candidate scores, not directions. In Python, $F[i, j]$ aligns `x[:i]` and `y[:j]`; their last letters are `x[i-1]` and `y[j-1]` when $i, j > 0$. Then give two optimal alignments for `AA` and `A`, and identify which one your tie rule returns.

```
allocate score table F and pointer table P
F[0,0] = ___; P[0,0] = None
for i = 1, ..., m: F[i,0] = ___; P[i,0] = ___
for j = 1, ..., n: F[0,j] = ___; P[0,j] = ___
for i = 1, ..., m:
    for j = 1, ..., n:
        cD = ___; cU = ___; cL = ___
        F[i,j] = ___
        P[i,j] = ___ # store a direction; apply tie rule
i, j = m, n
ax, ay = empty lists
while ___:
    if P[i,j] == "D": append ___ to ax, ay; set (i,j) = ___
    else if P[i,j] == "U": append ___ to ax, ay; set (i,j) = ___
    else: append ___ to ax, ay; set (i,j) = ___
reverse ax and ay once; join each list into a string
return ___
```

(d) Prove correctness and termination.

1. State the induction hypothesis for prefix pairs with smaller $i + j$, and verify the empty-prefix boundaries.
2. Remove the last column of any alignment. Use the induction hypothesis to bound its score by one recurrence candidate, and hence by their maximum.
3. Show that the maximum is attainable by extending an optimal alignment of the corresponding smaller prefixes.
4. Prove that traceback returns a legal alignment with score $F[m, n]$. Give a decreasing nonnegative integer for traceback and explain why the fill loops terminate, including when one input is empty.

(e) Analyze the algorithm. Count each table access, addition and comparison as constant time. Count interior updates and table entries, and bound the traceback length. Derive time and extra space, including empty inputs and output. Give the exact change in interior updates and the limiting table-entry ratio when both lengths double and when only n doubles for fixed $m > 0$. Explain why keeping only two score rows and no pointer table does not support this traceback.

Problem 4. Sequence alignment: implementation and measurement

25 points: five parts, 5 points each

(a) Implement and test. Implement `needleman_wunsch(x, y, counts=None)`, returning the score and both aligned strings. Use the seven input pairs in the table; `""` denotes an empty string. Fill expected scores before running, then enter the returned scores and aligned strings. For each output, check equal length, input recovery, no gap–gap column, and agreement with a separately recomputed score. For `AA/A`, also compare the strings with the tie-rule output you derived in Problem 3(c). Enter pass only if all applicable checks and the expected-score comparison pass; explain any failure below the table.

x	*y*	Expected score	Returned score	Aligned *x* / aligned *y*	Checks pass/fail
`""`	`""`	_____	_____	_____/ _____	_____
`""`	A	_____	_____	_____/ _____	_____
A	`""`	_____	_____	_____/ _____	_____
A	A	_____	_____	_____/ _____	_____
A	C	_____	_____	_____/ _____	_____
ACG	AG	_____	_____	_____/ _____	_____
AA	A	_____	_____	_____/ _____	_____

(b) Count work and storage. Increment `cells` for each interior update and `traceback` for each emitted column. Counted and uncounted runs must agree. For `ACG/AG`, report both counters and the total entries in the two tables in the supplied form. Explain which work the cell counter omits.

Quantity	Observed count
`cells`	_____
`traceback`	_____
Total entries in both tables	_____

Problem 4, continued: measurements

(c) Construct sequence families. For $n = 32, 64, 128, 256$, compare $m = n$ with fixed $m = 32$. Use the supplied `make_sequences(m, n, seed=527)` unchanged to generate each pair once. Each call resets the seed, then draws the m letters of x followed by the n letters of y, independently and uniformly from `ACGT`. Complete the table with predicted / observed cell counts, observed traceback moves, and the combined entries in both tables. Validate and rescore each output before timing. State which count ratios are exact, which have limits and which depend on sequence content.

Regime	m	n	Cells pred. / obs.	Traceback moves	Total table entries
square	32	32	_____/_____	_____	_____
square	64	64	_____/_____	_____	_____
square	128	128	_____/_____	_____	_____
square	256	256	_____/_____	_____	_____
fixed m	32	32	_____/_____	_____	_____
fixed m	32	64	_____/_____	_____	_____
fixed m	32	128	_____/_____	_____	_____
fixed m	32	256	_____/_____	_____	_____

(d) Measure the complete call. Time allocation, initialization, table fill, traceback and returned strings. Exclude input generation, validation, printing and plotting. Use the timing protocol from Problem 2(d) and fill the supplied timing table. Reuse the same fixed sequence pair for all calls at a given size. Compute ratios within each regime. Plot n on the horizontal axis and seconds per complete call on the vertical axis, with logarithmic scales and a labeled line for each regime.

Regime	m	n	$T(n)$ (seconds/call)	$T(n)/T(n/2)$
square	32	32	_____	—
square	64	64	_____	_____
square	128	128	_____	_____
square	256	256	_____	_____
fixed m	32	32	_____	—
fixed m	32	64	_____	_____
fixed m	32	128	_____	_____
fixed m	32	256	_____	_____

(e) Interpret the results. Use the doubling ratios from part (d) and compare them with Problem 3(e). Use table entries and traceback length to describe space; distinguish entries from bytes. Explain one difference between count and timing trends. State why timing does not prove correctness or a worst-case bound. Do repeated batches measure timing variability on one input or variability across random inputs? In two sentences, explain why the shortest-path algorithm can finalize one tentative label, whereas alignment compares all three last-column cases instead of greedily matching the next letters.